

Full Project Roadmap: Django + DRF Hybrid App

1. Project Setup

- Create a new Django project using ``django-admin startproject``.
- Create separate apps for frontend and API logic (e.g., ``core``, ``api``, ``web``).
- Install Django REST Framework and add it to ``INSTALLED_APPS``.
- Set up static files, templates, and configure base directory structure.
- Configure PostgreSQL or SQLite database (based on your preference).

2. User Authentication

- Use Django's built-in auth system for login/logout/registration.
- Set up user roles if needed (e.g., admin, user, moderator).
- Secure DRF endpoints using session/cookie-based authentication or token auth.
- Protect API views with permissions (e.g., `IsAuthenticated`, `IsAdminUser`).

3. Backend (DRF API Layer)

- Design your data models (e.g., Patient, Doctor, Appointment).
- Create serializers for each model.
- Build ViewSets and use routers to register API endpoints.
- Test endpoints using DRF's browsable API or Postman.
- Implement filtering, searching, and pagination if needed.

4. Frontend (Django Template Layer)

- Create base templates and extend them for each page (e.g., dashboard, profile).
- Use Django views (`TemplateView` or custom views) to render these templates.
- Inject essential context (user info, CSRF token) into templates.
- Use JavaScript (fetch or axios) inside templates to call your DRF endpoints.
- Render data dynamically in the DOM using vanilla JS or a minimal JS framework.

5. Features & Logic

- Add dynamic forms using AJAX (e.g., submit new record without page reload).
- Implement real-time updates or polling if needed (e.g., live notifications).
- Allow CRUD operations via both the API and template views.
- Use Django messages framework or JS notifications for user feedback.

6. Testing & Validation

- Write unit tests for your models and API views.
- Use Django's `TestCase` and DRF's `APIClient`.
- Manually test templates and JS functionality.
- Fix all UI/UX issues and broken workflows.

7. Deployment

- Collect static files with `python manage.py collectstatic`.
- Use Gunicorn + Nginx or deploy to platforms like Heroku, Render, or Railway.
- Set environment variables securely (e.g., SECRET_KEY, DB credentials).
- Ensure HTTPS, database backups, and logging are in place.